CONCURRENCY TESTING

Web and mobile applications enhance usability by providing easy ways to make updates, such as dragging and dropping items. When multiple users are looking at the same view of data, we need to verify that when one user makes a change, the other users see that change. If two users update the same piece of data at the same time, who "wins"?

Lisa's Story

The ability for one person to see updates made by another user in real time is an important feature of our product. For our new autosave feature, where changes to an input field should persist as soon as focus moves out of the field, testing concurrent changes was a must. We recognized this as a high-risk area, since users want to feel certain their changes are persisted. We had automated regression tests for concurrent updates but wanted to explore this area in more detail because we weren't feeling confident about the feature quality.

We scheduled what our team calls a "group hug," where multiple team members test at the same time, as described in Chapter 12, "Exploratory Testing." We time-boxed this session to one hour and wrote up test charters and scenarios in advance. These charters captured normal use as well as extreme worst-case scenarios.

We typed notes on the bugs we found and other observations in a shared document. Learning about a bug that one person found often inspired me to think up another test to try.

Here's a sampling of issues we noted that would be harder to find while testing individually:

- Changing an epic label when someone else has the epic open, the user who changes sees an error and the label blanks out.
- Starting or moving a story with someone else updating the description causes overwriting and disappearing history.

One interesting discovery was that we couldn't reliably reproduce some problems, which pointed to timing issues. We marked it to explore later to see if there was an issue in the implementation or in the way we tested. It turned out to be the tip of the iceberg of a bad bug that spurred a major code design change.

The group hug confirmed our suspicions that the feature still needed a lot of work. Our team needed to do some redesign, coding changes, and more exploring before we could consider releasing the feature for beta test.

During the group hug, we realized we still had questions about the desired behavior of the autosave feature. This led to further discussions within the

whole team, and within a few days, design improvements and development stories were under way. We knew that there would be more concurrency testing group hugs for this feature.

If your product risks losing updates when simultaneous updates occur or when users need to see updates by other users in real time, concurrency is an important area to think about before coding, but also to cover in your exploratory testing. Solutions for this type of requirement often involve caching updates, which can lead to both performance and transaction integrity issues. Automated regression tests for these scenarios are essential because it is difficult to test them manually; however, timing issues can be subtle and hard to find. Experiment with creative ways to test your features in the same way that customers will use them in production. Take advantage of regular events; for example, check your system's response time while the Olympics are being streamed live.

INTERNATIONALIZATION AND LOCALIZATION

With global markets come global challenges. Many organizations must support multiple languages due to changing business needs, and that creates challenges for agile teams. In traditional projects, the application is built, and then the strings that are to be translated are sent off to experts. The translators have the full context of the application, so the translations are generally correct. In agile teams, we have quick releases with the possibility of releasing to the customer every iteration or perhaps even continuously. Traditional methods do not work in that environment.

All Ÿøur βugs βeløñg tø Us

Paul Carvalho, *a test specialist from Ontario, Canada, shares his experiences with internationalization and localization testing in an agile way.*

A Roman walks into a bar, holds up two fingers, and says, "Five beers, please." *Ba-dum-cha.* This old joke relies on knowledge of a culture that is not your own. If you already know the answer to the joke, are

interested in finding out, or are looking for new cool ways to bring value to your software systems, internationalization (i18n) and localization (L10n) testing may be right for you.

I'll start with a couple of definitions. i18n lays the groundwork or framework for software and systems to be used in a particular language or region. L10n testing (on a properly internationalized base product) involves checking that the system is translated correctly and looks and works as expected according to some language or culture.

In simpler terms, i18n means the system is *ready to be* translated into another specific language. It's like having a box of Legos with the right pieces to build something—like a fire truck or *Star Wars* set. Meanwhile, L10n means the system *is* translated into another language. Continuing the Lego analogy, you have now built the Lego set into the desired product for someone.

Think about the end product and users' desires. You don't expect a *Star Wars* set to satisfy a medieval castle fan. While there might be some similar pieces, you will need to reexamine the requirements and put the fundamental (i18n) pieces into the box to be successful with different audiences.

This example illustrates one of the advantages to developing internationalized systems in agile ways. An agile team focuses on delivering working software frequently and satisfying the customer through early delivery of valuable software. That is, don't try to solve all your i18n and L10n problems at once. Make different customers happy as you go.

As a team, before you start building anything, get together and come up with specific examples of how the system should look and work for people in a particular target region or country. Remember the Lego example—BEWARE OF ASSUMPTIONS. Not all languages are the same everywhere.

Take English, for example. In my travels, I have heard English spoken, spelled, and used in many different ways. English from England (EN-GB) is different from English in the United States (EN-US) is different from English in Australia (EN-AU). If you say your software supports English, I will ask, "Which one(s)?" For example, if a child asks for a *Star Wars* Lego set for his birthday, you'd better be sure you know what that means to him. There is a big difference between an Ewok village set and a *Millennium Falcon*. Guessing wrong may lead down the path to suffering.

Use an iterative approach to building localized systems. For instance, in the first iteration, you may settle on EN-US (if you work for an American company or market) in order to complete a particular feature that provides some value to your target market. When you complete that feature and decide to take on your next target region or audience, always make sure you consult with people who are local to that particular region. You are *loco* (aka crazy, aka nuts, aka . . .) if you don't use locals for localizations.

For example, let's say a U.S. company wants to localize software for its Canadian neighbors (EN-CA). A Canadian could easily tell you that there are more than just spelling differences in some of the words. Yes, the dictionary of terms for labels and outputs is an important piece. However, units of measure require more than just label changes.

Once, before a presentation on i18n and L10n, I used a web-based translation program to translate the following sentence from English to French:

> **Before:** "We need **40,000 lbs** of fuel."
>
> **After:** "Nous avons besoin de **40.000 kg** de carburant."

Eek! No! Translation requires conversions, too! Let's say it's a hot day and your EN-US weather app says it's 100˚F outside. If the user switches to EN-CA, changing *only* the unit label to now say it is 100˚C is so many levels of wrong. "People are dying" kind of wrong. We never want to make mistakes that threaten people's lives.

So now you need to add code to your software that takes an input value and makes the appropriate conversion before applying the appropriate unit label. This is where test-driven development allows you to safely make changes to the system to account for new and changing requirements while remembering the requirements on the base features.

Internationalization and localization need to be designed into the system. As with other important quality requirements, we iterate on the design and seek feedback frequently. To be successful, you need access to a specialist on your team, just as with security, performance, or user experience.

Some languages and regions have specific needs that you might never guess on your own, such as capitalization rules or name sorting. Even when translating into European languages that use the same base character set, you need more than just support for the extended ASCII character set.

Does the delivery team need to immerse themselves in a particular culture to be successful? I think it might be fun to bring the team closer to their target audience by getting out of the office and exploring multicultural multimedia opportunities to enrich their appreciation for other cultures and regions.

I'm convinced that Hawaiian English needs more exploration and that my employer should support a research expedition. With my luck, I'll probably get a DVD copy of the Elvis movie *Blue Hawaii*.

If your product has a global clientele, do what you can to avoid frustrating customers who use other languages and character sets. Support globalization (g11n), which includes internationalization and localization, with tests that guide development, exploratory testing, and perhaps other types of testing, such as linguistic testing. Internationalization is a constraint that developers must consider as they are coding and incorporate into every story. For example, it would include encoding, formatting, and externalizing strings across the code base. If you are working on legacy code, perhaps you have stories specifically to address some of the i18n requirements.

Localization is the translation piece, not only of the language itself, but often of nuances for specific locales. Terminology needs to be established, but perhaps all of it doesn't need to be done up front. The point when teams break features into stories might be a good time to think of the terms that should be used. Maybe the first story is for an analyst to determine what words will be used for consistency across the product. Frameworks to support localizing and translating can speed up the process, but they don't eliminate the need to verify the translations and suitability for specific cultures and regions.

In Figure 13-1, Lisa shows the terminology for the parts of a donkey harness. To Janet, it seems like a foreign language, but at least now we have common terms and can point to the same piece of harness and mean the same thing. Even writing this book, we had to compromise between EN-US and EN-CA spellings. Since our audience is global, we tried to avoid using slang and colloquialisms.

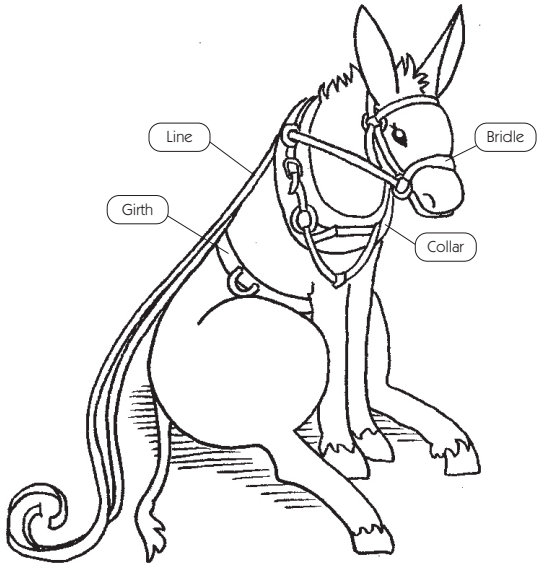


Figure 13-1 Donkey harness vocabulary

Your organizational culture may have a large impact on how you approach localization on your team. As Paul Carvalho mentions in his story, having local support for languages is ideal. However, that may not always be possible, since it is expensive to have many experts available at all times for translations. There are alternative solutions that may allow you to take advantage of fast (or faster) feedback than is usual on a phased-and-gated-style project. Perhaps you can use machine translations, which are getting better all the time. The trade-off is the need for significant post-editing to make sure you catch the errors in the automated conversions.

Another approach might be a hybrid, where agile teams create a "drop" for translators. This drop would include a whole feature, rather than specific strings to translate from each story, and would allow translators to have some context for their translations. If your release cycle is six months long, perhaps a drop of every four weeks could give you fast enough feedback to correct any mistakes before the end game, while minimizing the overhead of translations. Figure 13-2 shows this alternative. It is all about balancing speed with quality, time, and functionality.

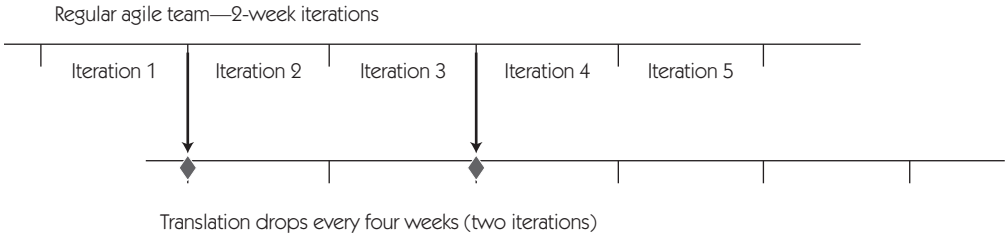


Figure 13-2 Possible schedule for translation drops

Remember, if you are using third-party vendors, have agreed-upon time frames and expectations, and take advantage of automation in file drops for consistency in the process.

It is a different mindset to embrace rapid release cycles, and we encourage teams to think about how they can make changes to their process to get fast feedback. Perhaps, just perhaps, the local experts exist on some of your global teams.

REGRESSION TESTING CHALLENGES

First, we give a quick definition of what we mean by regression testing because it can be a contentious term. To us, regression tests are those tests that run regularly to give you confidence that changes made to the code do not affect existing functionality unexpectedly. We believe that automated checks can do this with the fastest feedback. At a minimum, these should run nightly.

Now that many teams release once a week, several times per week, or several times per day, regression testing is an even bigger challenge. Even if you automate your regression tests, they may not all run fast enough to complete in time for the next release.

Many companies faced with this problem turn to "testing in production." Seth Eliot has written and presented extensively on this subject. He defines testing in production this way (Eliot, 2012):

> *Testing in production (TiP) is a set of software testing methodologies that utilizes real users and production environments in a way that both*

> *leverages the diversity of production, while mitigating risks to end users. By leveraging the diversity of production we are able to exercise code paths and use cases that we were unable to achieve in our test lab, or did not anticipate in our test planning.*

Lisa's team has good coverage from automated test suites running in the continuous integration system at all levels: unit, functional, and user interface (UI). They also spend lots of time doing exploratory testing, but it's hard to cover every scenario. For new major versions, they use a TiP approach. They enable the new version for a small percentage of users and monitor production logs to see if users are experiencing errors. As they identify new defects, they fix them as needed. They have a rollback plan to disable the feature if there are unacceptable results. When the new features appear to be stable, the team enables them for more users, continuing to watch logs carefully, until all users are able to use them.

As much as we automate regression tests, there may be tests that are difficult to automate in a way that provides proper feedback, and other tests that are too costly to automate. It's also a challenge to do manual regression tests for quality attributes such as look and feel when releasing so frequently. The testers on Lisa's team created wiki pages with manual regression test checklists for different areas of the product based on risk. Often, programmers use the checklists to do the manual regression testing, or at least help with it. The team regularly audits the checklists to see if any tests can be or have been automated. They keep the lists as concise as possible, focused on the high-risk areas. In Chapter 12, "Exploratory Testing," there were a couple of examples of managing regression testing with session-based test management or thread-based test management if you do not have automation. How you manage your regression suite will depend on your context, the risks within your system, and how often you release to production.

User Acceptance Testing

User acceptance testing (UAT) is part of Quadrant 3, business-facing tests that critique the product. We describe user acceptance testing as "making sure the actual users can do their job." UAT may be performed by product managers, but preferably it's done by the actual end users of

the product. As we explained in *Agile Testing*, UAT is often done as part of a post-development testing cycle (pp. 467–68), but this does not mean it has to be left to the end game. Janet encourages companies she works with to think of ways to bring it earlier into the development cycle.

Janet's Story

I started with a team that was on three-month delivery cycles, which should have meant that they released to the customer every three months. However, the UAT testing took another six weeks, so new features weren't actually in production for almost six months. I found out that the reason for such a long UAT cycle was that the person (I'll call her Betty) doing the testing had to do it in addition to her own job. She had to "fit it in" around her regular duties, and six weeks was how long it took.

I suggested that we have Betty sit with the team at the end of each two-week iteration and play with the new features we were developing. I paired with her for a while to show her the features and then let her be. At the end of the three months when we were ready to deliver the new features, Betty asked for only three weeks (instead of the usual six) for UAT.

This was a substantial improvement, but I wanted it to be even better. We got her a workstation in the team work area, and Betty started coming in every Friday afternoon. She gained confidence in our work and what we were delivering. At the end of that release, we included one full dedicated day of UAT and were able to put the release into production within the three-month delivery cycle.

Lisa's current team develops a software-as-a-service (SaaS) product that is also used internally by the entire company. This provides a great opportunity to release new features for internal-only beta and get feedback from actual users who happen to be in the same company. Problems with real-life use are identified and fixed before the new features are made available to paying customers.

Understand your customers, your real users, and brainstorm ways to get them to use the system to make sure they can do their job and use the system appropriately. Customers are the ultimate judges of software value. End users are probably the best people to critique your product.

A/B Testing

A/B or split testing is often used in lean startup products or in existing products that are changing their look. It is a different type of testing in that it validates a business idea, so in some ways it is a Q2 type of test. However, it is done in production by real customers, so it's really a way of critiquing the product.

The idea is to develop two distinct implementations, each representing a different hypothesis about user behavior, and put them out for production customers to use. For example, you can move UI elements around or change the steps of a UI wizard. The company monitors statistics on which customers "click through" and which leave right away. In this way, companies can base decisions on real results and can improve their applications based on continued A/B experiments. When appropriately done, A/B testing can help companies make decisions about everything from user experience (UX) design to pricing plans.

Small Changes Can Have Big Impacts

Toby Sinclair, *a tester in the UK, shares his experiences at a retail company that did extensive A/B testing.*

Whilst I worked with a large online UK retailer in 2011, they introduced the concept of A/B testing. The usability and customer experience teams wanted to have more confidence in the design decisions being made, and they saw A/B testing as a way to do this.

The initial tool used was Campaign Optimizer, as it integrated with our existing Oracle ATG e-commerce platform. It allowed the business to set up A/B tests without code deployments. This meant they could easily switch tests on and off without contacting the development teams. The tool also provides the administration facility (reporting, switching tests on/off, etc.). Some initial development and integration testing was required to get it up and running.

The first A/B tests the team implemented were very simple, for example, a 50/50 split between these two choices:

- **Existing version:** 20 items on shopping gallery page by default
- **The challenger:** 50 items on shopping gallery page by default

The winner was decided based upon a number of metrics. These included

- Pages viewed by a user
- Particular items on a page viewed by a user
- Products added to a shopping cart by a user
- Purchases made by a user

Deciding how to split your tests and how long to run them is quite an art. Some of the factors include:

- How many visits did you get to your site? You need a high count in order to have confidence in your results.
- How experimental is the challenger? You may want to send only a small number of customers to the challenger if the impact is unknown.

Generally most of our A/B tests ran for two weeks, as we found that gave us a good statistical result based upon the average customer visits. To validate the results from the A/B tests, we also compared information from our analytics partners (Coremetrics and Speed-Trap). This ensured that we had full confidence in the winning version.

We also developed the capability to run tests against specific customer segments. This made the results obtained even richer and provided further insights to the targeted content team.

Today, every day, there are tests running across the website providing rich information to the teams about how best to design the website. The capabilities of and focus on A/B testing have grown considerably over the past few years. There is now even a separate A/B test team that recruits people with both a technical and marketing background.

A/B testing is definitely not limited to agile projects, but it fits agile values well. The goal of A/B testing is to identify which changes to your website will have the greatest impact. This type of testing is all about iterating with fast feedback to select a design or achieve and maintain other quality characteristics that help the business achieve its goals. If you think your team might benefit from A/B testing, check the links in the bibliography for Part V to learn more.

User Experience Testing

User experience (UX) designers have many ways to get feedback on functional website design and designs in progress. They use methods commonly found in industrial design and anthropology to test designs and design concepts before producing anything on screen or even on paper.

User Research

Drew McKinney, *a product designer on Lisa's current team, shares some of his experiences with user research and user experience testing.*

One of the projects I led was to develop a project planning and management system for Disney Animation Studios artists and technical staff. My team conducted ethnographic studies with software developers, technical directors, technology managers, and animation technologists to understand their collaborative work processes. In a process known as contextual inquiry, the designer or researcher observes the user perform day-to-day activities and discusses them with the user while taking part (Wikipedia, 2014b).

In our case, these predesign sessions revealed several problems with the artist software delivery system and technical project management. Ongoing communication issues coupled with a poorly designed software indexing system resulted in numerous headaches for both technical and artist staffs. As a result of these early predesign exercises, it was easier for our team to design an appropriate solution to Disney's software (and people) management problem.

Prototype Testing and Paper Prototypes

Design testing can be a lengthy process if done incorrectly. Testing too late in the process can reveal fundamental flaws with a new system's design, requiring costly redesigns and rewrites. To provide early feedback on product designs, UX designers use a number of prototyping methods throughout the product development process. These prototypes can range from full-color printouts to simple sketches.

The easiest way to achieve quick usability feedback is also the cheapest: paper prototypes. Paper prototypes are paper representations of interfaces. On a different project, my team designed an iPhone app for a weight-loss program. Again, we used early paper prototypes to see how users would respond to an app concept. We noticed that

users were confused about the meaning of representations on a map, so we improved the graphical design to make the meaning clearer. We observed that the iPhone users were more attracted to applications that provided a fun and fresh experience, so we designed their system to accommodate that need.

Sometimes paper prototypes are not enough, and a more functional example needs to be used. In another example from Disney, our team was tasked with building a collaborative canvas interface to allow senior animators to critique remotely located artists, a process that still relied on fax machines and phone calls. This was a clear opportunity for a functional prototype: a working example that artists could use as if it were final. Rather than build a prototype, we used off-the-shelf hardware and software to accomplish this. We attached a Teradici remote workstation to a Cintiq monitor and placed a MacBook above the workspace with a FaceTime call to the other artist. While by no means a final product, this quick prototype allowed us to vet the design to understand the benefits and limitations of such a system. Two Disney animators worked together remotely to test a design prototype. Figure 13-3 is a representation of this collaboration.

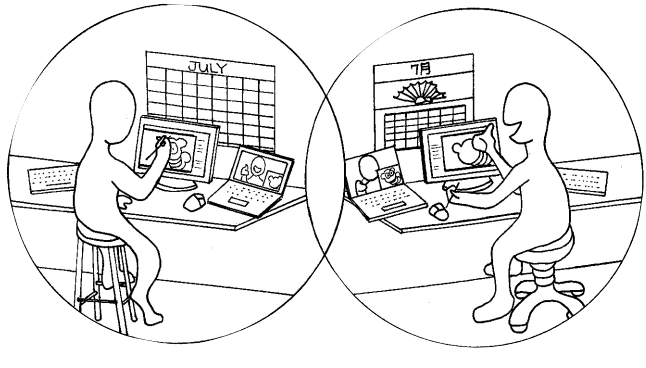


Figure 13-3 UX animators collaborating remotely

User experience testing is an example of a type of testing that can fit into more than one of the agile testing quadrants. You can get feedback from users before any coding is started, using paper prototypes and other techniques such as the ones Drew describes. This is a form of Quadrant 2 testing, creating customer-facing examples and information

that will guide development. You can also do usability testing after coding is complete or monitor the product already in production to learn whether the current design and functionality are adequate. Those are examples of Quadrant 3 activities, evaluating the software from a business and user perspective. Feedback from this may result in new features and stories to be done in the future.

Nordstrom Labs recorded its in-store innovation testing efforts for an iPad app that would help customers choose eyeglass frames (Nordstrom, 2011). The video shows testing activities ongoing throughout iterations that lasted minutes rather than days. Designers and testers on Lisa's team have done usability testing with both internal company users of a product and people outside the company, taking advantage of user group meet-ups. Look for ways to test with real end users rather than speculate about how they will use a particular feature.

Sit with your product's existing users and see where they struggle. Take your new design to your local coffee shop and see what people think of it. Involve current and desired customers early and often. Chapter 20, "Agile Testing for Mobile and Embedded Systems," has a bit more on user experience but focuses more on how it relates to mobile apps. Check the Part V bibliography for more links on these different types of testing.

Summary

Don't repeat the mistakes of our early XP/agile teams and focus exclusively on functional testing. In this chapter, we discussed some of the types of testing that are outside the scope of what is generally known as functional tests. Many of these tests can be done with an exploratory approach.

- Use the Quadrants to think about all the different types of testing your product requires.
- Talk with your business stakeholders to learn their expectations for attributes such as stability, performance, security, usability, and other "ilities."
- Concurrency testing is key for products whose users may update the same data simultaneously. Use both automated and

exploratory tests to ensure that updates are reflected correctly and in a timely manner.

- Internationalization and localization testing requires looking at cultural differences as well as languages and character sets. Specialists in this field are needed, just as many software products require security, performance, or UX testing experts.
- Testing (monitoring) in production is one approach to finding defects that are missed by checking and exploring during development.
- Completing user acceptance testing during development, rather than after, shortens the UAT cycle needed during the prerelease end game.
- A/B testing is one way to get fast feedback from production users about aspects of the application design, using a series of experiments, each one building on what was learned from the last.
- Usability testing can be done simply and productively with paper prototypes and conversations with users to help your team refine your designs and features.

Part VI

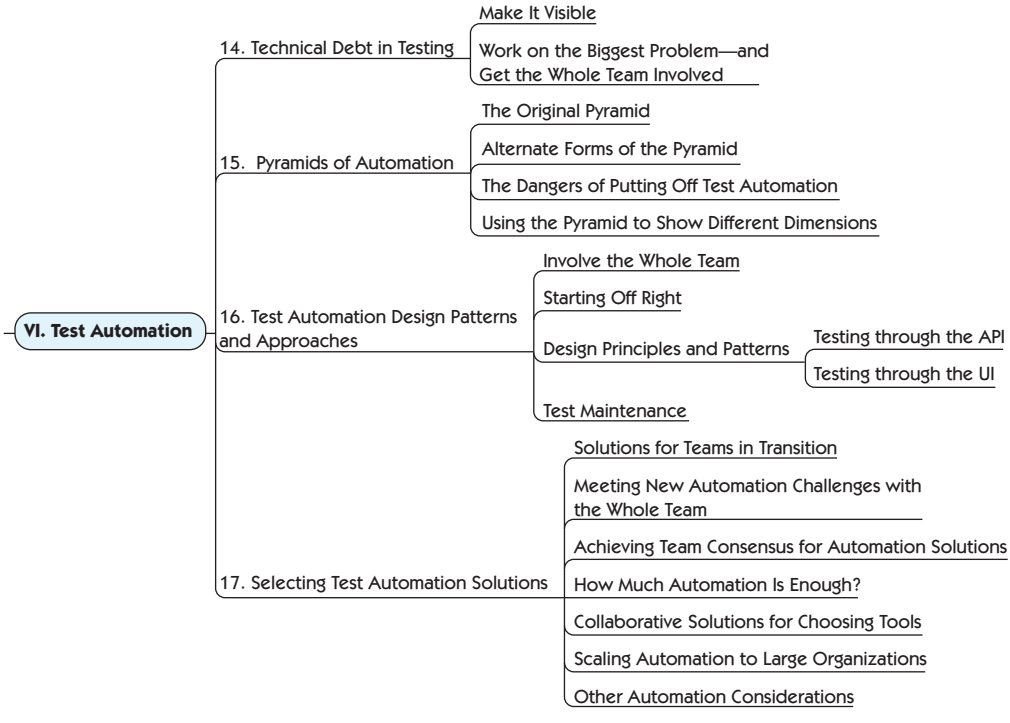
Test Automation

Test automation is necessary. We need to automate to manage technical debt, make time for critical testing activities such as exploratory testing, and guide development with customer-facing examples. Automated regression tests tell us quickly if we've broken existing production code. As well, our automated tests provide excellent living documentation for our application.

Test automation is hard. Some types of automation are fairly painless and can even be rewarding once you get past the "hump of pain" of learning how. Some present ongoing challenges, such as testing JavaScript and logic-heavy user interface pages and experiencing sporadic failures due to timing or automatic browser upgrades.

Agile Testing included a large section about automation. To learn more about the reasons to automate tests, barriers that get in the way of automation, and how to put together an agile test automation strategy based on the test automation pyramid and agile testing quadrants, please refer to it.

Here in Part VI, we will explore more ways that testers and agile teams can succeed over the long term in creating maintainable test automation. Automated tests can help teams achieve a sustainable pace with manageable levels of technical debt. But this works only if your tests provide quick feedback at a low-enough cost. We'll look at alternative interpretations of the test automation pyramid and how those can inspire us to do more experiments involving the whole team in solving automation problems.



- **Chapter 14**, "Technical Debt in Testing"
- **Chapter 15**, "Pyramids of Automation"
- **Chapter 16**, "Test Automation Design Patterns and Approaches"
- **Chapter 17**, "Selecting Test Automation Solutions"

Chapter 14

Technical Debt in Testing

Ward Cunningham coined the term *technical debt* (Cunningham, 2009) to represent rushing delivery of a feature or user story without ever going back to refactor the code to express your learnings. We can incur crippling technical debt in our automated test code as well as our production code. Teams often make mistakes in their initial attempts at functional test automation partly because today's frameworks and drivers make it easy to automate tests quickly. However, if we don't pay as much attention to code design principles and patterns with automated test code as we do (ideally) with production code, maintaining test scripts may become a burden. It's tricky to refactor automated test code; is a test failure a mistake in the refactoring or an actual regression bug in the software? One of the first references to technical debt in testing that we found is Markus Gärtner's blog post "Technical Debt Applied to Testing" (Gärtner, 2009).

If test automation is inadequate, or if automated tests take too much time for maintenance, there's less time for other crucial testing activities. If time is short, teams give in to temptation and skip test activities from Quadrants 3 and 4. (See Chapter 8, "Using Models to Help Plan," for more on the agile testing quadrants.) If the team feels pressure to meet a deadline, and automated tests are failing, they may rationalize that they need to fix the production code, but fixing the tests can be put off until later. When testers are consistently playing catch-up by testing code that was checked in a prior iteration, they won't have time to collaborate with the customers to elicit examples and specify business-facing tests to guide development for the current stories. This becomes a vicious cycle; no time, so we skip important testing activities, which

results in more regression failures and less likelihood that we will build the right features for the customer. When testing and coding are equally valued, we can avoid this pitfall.

You can do a pretty good job of automating tests at the different levels of the test automation pyramid (see Chapter 15, "Pyramids of Automation") and still find that it wasn't enough. As a product changes, some teams find that it gets harder to update the automated regression tests accordingly. As with production code, test code might be hard to read and understand or contain unnecessary duplication. It needs the same care and feeding as production code.

Let's look at ways to manage technical debt in your testing.

MAKE IT VISIBLE

As with many impediments that agile teams face, you can start cutting your team's test technical debt down to a manageable level by first giving it visibility. For every story in the backlog that involves changing code, think about what existing automated tests might be affected. If necessary, take a quick look at the automated tests to get an idea of the possible effort needed. Make sure the estimate reflects the time needed to update the tests, and write a task card for doing so (see Figure 14-1).

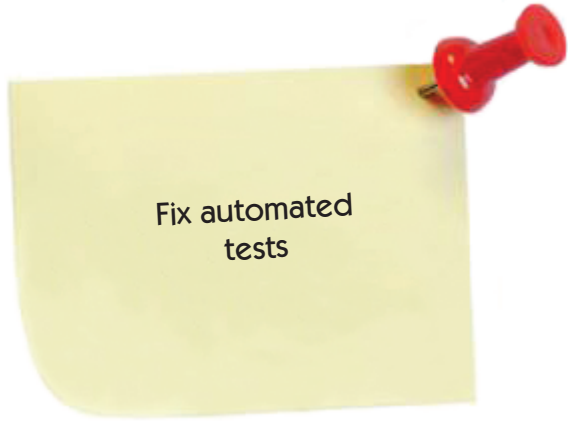


Figure 14-1 Task example for addressing existing automated tests

Write stories for any large test code refactoring that you need to do, just as you would for production code.

Track how much time you spend on test maintenance, and make it visible. See where you're spending your time. Is it on debugging tests that fail, or updating tests that give false results or were otherwise poorly designed? Perhaps you can write task or story cards for each failure that has to be investigated and for maintaining or refactoring test scripts to create visibility.

Similarly, make sure estimates and tasks reflect the amount of effort that goes into manual regression testing of each release. Remember that the whole team is responsible for all testing activities, including regression testing. It is common for programmers and other team members to help with manual regression testing. We call this "sharing the pain," and it enables the team to make better decisions about how to move forward.

Reducing Defect Debt—Zero Tolerance

Augusto Evangelisti *shares how his team reduced their defects and gained more enjoyment.*

Disclaimer: This won't work for everybody; all I claim is, "It worked in my context," and you might try to use it at your own risk.

One day, a few years ago, I had a conversation with an inspirational man; he was my CTO at that time. He talked about zero tolerance of bugs and how beneficial it is to remove the annoyances of bug management from the development of software. I listened, but at that time I didn't grasp the concept completely; I had never seen or envisioned a situation where a development team could produce zero or for that matter even close to zero bugs. Since then, a few years have passed and I have worked hard on his vision. Today I can say that he was right; a delivery team can deliver value with zero known bugs. I even learned that the project team doesn't have to spend half of its time playing with a bug management tool to file, prioritize, and shift waste.

Let me give you some context around my project and current practices. We have two colocated, cross-functional agile teams, each with four developers, one tester, one business analyst, one product owner, one DevOps guy, and a team lead who functions as kanban facilitator, for a total of 18 people.

Figure 14-2 Augusto's kanban board

We use kanban to visualize our process, and we have the ability to deliver every time we complete a user story if we want. Our kanban board (see Figure 14-2) visualizes our process. In the top center, there's an "ATDD" (acceptance-test-driven development) section with column headers borrowed from Elisabeth Hendrickson's work: "Discuss," "Distill," "Develop," and "Demo."

We keep our user stories small, so that we can deliver them in less than three days. The stories are vertical slices, meaning that no integration is required between teams. Each team works on the full code base, which spans multiple applications.

We use a hybrid of ATDD and specification by example (SBE) [*authors' note:* see Chapter 11, "Getting Examples," for more on those], which I've specifically developed to fit our context. Programmers code-review every line of code before every deployment; they also practice some pair programming and test-driven development (TDD) (Hendrickson, 2008).

We automate close to 100% of our acceptance tests, using jBehave and Thucydides. See Figure 14-3 for a visual of our ATDD cycle. It's based on a model by James Shore, with changes suggested by Grigori Melnick, Brian Marick, and Elisabeth Hendrickson, and the SBE concept from Gojko Adzic. It also contains the "Red, Green, Refactor" image from *Agile in a Flash* (Ottinger and Langr, 2009c).

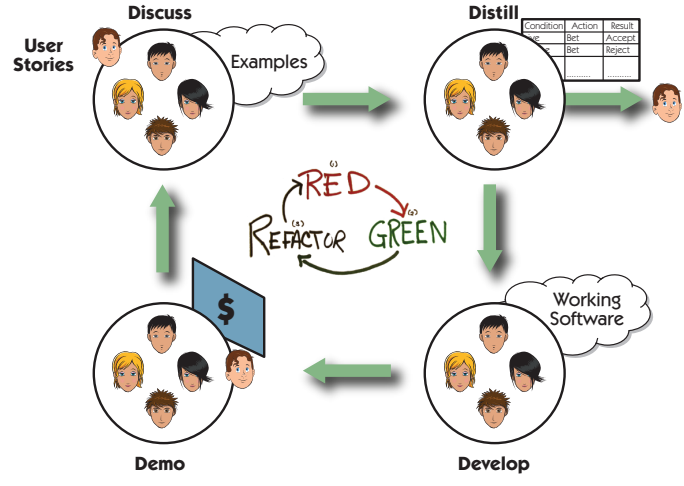


Figure 14-3 ATDD model used in Paddy Power

We have an internally developed "platform-on-demand-style" build and deployment pipeline that runs all our functional tests, as well as performance/load tests that check variations against a known baseline. Every automated test runs after every push to trunk in the pipeline.

We perform exploratory testing on a story once all the automated acceptance tests pass. Once exploratory testing is complete, we demo to our product owners, who subsequently do user acceptance testing (UAT) before accepting the story.

If a bug is found during exploratory testing or user acceptance testing, the programmer(s) who worked on that user story drops everything else he or she might be doing and fixes the bug immediately. The card will *not* progress if a valid bug is not fixed. The build must always be green. If it fails (turns red), the developer who caused the instability drops everything else and fixes the issue straightaway.

When a programmer fixes a bug, he or she writes automated tests that cover its path. You may have noticed that I've mentioned bugs. Yes, of course, we are humans and we make mistakes. This doesn't mean that we need to celebrate the mistakes and make them visible by logging them in bug-tracking tools or that we need to maintain them along with the waste stored in them, when the poor bug has the life span of an unlucky butterfly.

When I find a bug while doing exploratory testing, I simply go to the developer and say in a friendly way, "I think there might be an issue; come to my desk and I'll show you." If the programmer needs to go home and can't fix it straightaway, no problem; I stick a red sticky note on the user story card on the physical kanban board with two words describing the bug as a reminder. That card is not going anywhere until the bug is fixed and buried. The red sticky note goes into the trash bin after the death of the bug.

The development approach we follow allows us to have a good understanding of the business value we deliver because we have group discussions to derive examples for each user story. When you add a bunch of excellent developers who follow good engineering practices to that mix, you find out that the bugs you discover when exploring the software are very few. When you act upon them immediately, with no excuses, bugs don't really bother you.

In our situation, logging and managing bugs would simply be waste. We all live happily with no bug Ping-Ponging between programmers and testers, no bug prioritization or triage meetings, no bug statistics, and no need for bug trends to identify product release dates. And guess what? We deliver software that has no known bugs and delights our customers.

Augusto's team has found ways to build what their customers want as well as to prevent defects from finding their way to the customer. Part of their strategy is to put a red sticky note on the story card whenever they find a bug while testing the story, providing visibility. Everyone can see at a glance when there's a bug blocking the story. Make both testing activities and defects visible in your project-tracking system, whether it's a physical board on the wall with index cards or a virtual online board. With this visibility, everyone can see if the team is taking on too much work and running out of time for testing within an iteration. Carrying testing tasks over to the next iteration is sometimes unavoidable, but it may be a red flag that the team has built up too much testing technical debt, if not overall technical debt.

Janet's Story

In one organization I worked with, we had a large set of automated regression tests, but they started failing and taking longer and longer. We investigated and found out that the technical debt in that set of tests was

enormous. The tests were not only brittle and complex, but there were many that tested the same thing. That might be manageable in a waterfall process, but it becomes a burden in agile. No one had been watching or paying attention to test designs. We measured how much time was spent on maintenance and made a decision to have one person spend half days for a month to refactor to remove duplication and simplify the tests. This approach reduced the maintenance costs; not only was it easier to find and fix any problems, but it also reduced the time to run the tests and improved the quality and coverage of the tests. By making the problem visible, the team (including the product owner) was able to make a choice about how to fix it.

WORK ON THE BIGGEST PROBLEM—AND GET THE WHOLE TEAM INVOLVED

Once you have visual cues of technical debt, such as testing tasks left uncompleted at the end of the iteration, red (broken) continuous integration (CI) builds, or too much time being spent on test maintenance, take action. Test technical debt is something the whole team must manage. Remember that the whole team is responsible for all testing activities, from turning examples into automated tests that guide development to exploratory testing and regression checking.

Team retrospectives are a perfect opportunity to bring up areas of test technical debt that need to be repaid. The team can discuss contributing factors for each source of debt and use dot voting or a similar technique to identify the biggest problem that should be addressed first. Think of small experiments you could try to chip away at that problem.

For example, let's say your biggest testing problems are that the automated regression tests take too long to run, failures take a long time to pinpoint, and tests constantly get spurious false passes or failures (Janet calls these "flaky tests"). One way to speed up the builds is by adding build slaves on a virtual machine and multiple test suites in parallel. Flaky tests could be separated into a separate suite and could automatically be retried once before investigating the failure. They could be measured to see how often they fail or pass, and decisions could then be made about whether to fix or rewrite them so they could eventually be moved back into the regular regression suite. Maybe some tests are no longer worth keeping around, or they cover the same ground as lower-level automated

regression tests and need to be pruned out. Perhaps some unused test data could be removed so that setting up data for the tests goes more quickly. It might even be a more complex process of reviewing tests to see if some could be pruned out or if tests require refactoring.

As your team thinks of potential ways to address test technical debt, make the potential solutions visible, just as you did the problems. Write and estimate stories for activities to start eliminating your test technical debt. Lisa's team uses a separate project to track stories and chores (tasks) for things like reducing spurious CI job failures. They hold a short meeting each week, and high-priority stories and chores are moved into the team's active backlog as needed. If several experiments fail to make the problem better, a small cross-functional group brainstorms more solutions, and individuals take responsibility for trying different ones. They revisit the issues they have chosen to work on during each retrospective until they're no longer holding the team back.

Reducing Test Technical Debt through Collaboration

> **Chris George**, *a tester in the UK, tells us this story of how a tester and a developer joined forces to do what many thought was "impossible."*

The "impossible" task was turning around a legacy test suite from a 10% failure rate (just not always the same 10%!) in 8,000 tests that took over 12 hours to run, to 100% passing in significantly less time. There had been many attempts at fixing it with varying degrees of success, but other project work would usually interrupt any progress.

These tests had developed notoriety among the delivery teams, and the estimate for fixing the test suite was in the order of several months. Unfortunately, the tests could not simply be deleted because the code they covered was used by several products, one of which was moving to a frequent release strategy, and these tests were blocking its progress.

Jeff, one of the developers, and I decided that we would take a look at the tests. We had experience in improving test suites and figured we could apply our experiences to this one. We were given two weeks to see how much we could do. Previous attempts to fix the tests involved major rewrites of the underlying test framework. The approach we wanted to take was somewhat less invasive and did not involve a rewrite.

We started by each of us taking a small section of the failing tests, analyzing and categorizing the failures, then fixing the easy ones. Most

of the tests used databases and database backups, and it turned out that many of the failures were due to configuration issues. These were fairly easily fixed.

Over the course of two days, we fixed roughly 50% of the failures! Another two days, and surely we'd be finished!

As we investigated the remaining issues, we found that the tests were creating lots of databases but were not clearing up after themselves. We've both worked with SQL Server long enough to realize that the more databases you have on a server, the slower it gets. So even if a tenth of the total number of tests created one database each, this would really harm the server performance and add significant time to the test run.

If this had been left up to me, I would have put database drop commands in the test teardown. Thankfully, it was not left up to me, and in fact we paired on the problem. Jeff came up with the idea of applying the Dispose pattern to this problem, which is used to handle resource cleanup. We already created a database object when we restored a database, so we simply made this object disposable. The dispose method then called the drop command. By doing this, the author of the test does not need to remember to drop the database!

For the remainder of the week, Jeff and I applied this pattern to every test. Our rationale behind doing this was reducing the issues we knew about. Once we removed obvious causes of instability and failure, we would be left with a set of real failures to investigate.

As a break from fixing tests, we had a stab at profiling some of the long-running tests. This actually proved to be very lucrative indeed! We discovered that one of the main comparison routines for validating results used an O(N^2) algorithm! (For information on O notation, see Bell, 2014.) This was fine for a result set with only a few entries, but the time it took to run this comparison increased exponentially as the result sets grew! This was bonkers, and using Jeff's expertise, we refactored the routine to be simply O(N). This change alone knocked hours off the running time!

Some of the other quick wins were things like upgrading third-party libraries to the latest versions, and general "make-it-better" refactoring of the code, applying the Boy Scout rule (Kubasek, 2011) of leaving the campsite cleaner than we found it.

We fixed many more tests over the remainder of the week, some easy, some hard, and where we needed to, we paired. We used Jeff's development expertise, but we also leaned on my testing expertise to determine the usefulness of many of the tests.

At the end of the two weeks, the number of tests had actually increased to 9,000 (after we "found" 1,000 tests that were not being run because of a configuration error), but we had successfully reduced the failing tests to one, and now the average test suite run time per platform was down to around two hours.

This would not have been possible without the combined skills of Jeff and me. Having the two different perspectives and overlapping skill sets put us in a really good position to solve any issue we encountered. We also proved that an apparently impossible task was actually possible given a methodical and multipronged approach.

Chris's story shows the value of team members in different disciplines taking responsibility for managing test technical debt. The same is true when making the decision to incur this debt. There are times when it's appropriate to cut corners in order to meet a crucial business deadline, but make these decisions consciously, and involve testers, programmers, business stakeholders, DevOps staff, analysts, and other roles as appropriate. Explain to the business customers the future costs of deferring any kind of testing, and make sure they understand there will be more work later to rectify the shortcuts taken. Document the decisions in some way, perhaps via user stories for the future or on the team wiki, so the reasons for skipping some essential testing activity aren't lost in the mists of time—or worse, forgotten altogether.

SUMMARY

Teams may decide to incur test technical debt by deferring test automation or other testing activities in order to accommodate the business, but that debt has to be managed so it doesn't slow the team down over time. Inadequate test automation, in particular, often leads to spending more time on testing activities and causes testing to fall behind coding.

In this chapter, we looked at some ways to manage test technical debt, especially as it relates to test automation.

- Make the time spent debugging test failures and maintaining automated tests visible by writing task cards for these activities or reflecting the extra time in story estimates.
- Making bugs and the resulting blocked stories visible reminds the team to fix defects quickly, focus on defect prevention, and avoid incurring technical debt in the form of defect queues.
- The whole team needs to take responsibility for managing technical debt—both code and test.
- Identify the biggest source of technical debt, and try experiments to reduce that debt.
- Have team members with different specialties work together and apply those different skill sets to reduce technical debt.

This page intentionally left blank

Chapter 15

Pyramids of Automation

One of the questions we hear most often is, "How do we decide what to automate?" Our preferred model is the pyramid, although other models, such as the agile testing quadrants, can help as well. In the past few years, practitioners have adapted Mike Cohn's test automation pyramid that we presented in *Agile Testing*. We've seen some useful extensions, but the basic ideas behind the model stay the same. In this chapter, we'll explore the potential benefits of the newer interpretations.

The Original Pyramid

The basic principles of the test automation pyramid shown in Figure 15-1 still apply for most teams and projects, with most of the benefit based on how fast the feedback is received. The lowest level, the unit tests, gets the fastest feedback by running with every commit of new code.

We changed some wording on the pyramid to clarify its purpose. Testing business rules at the API layer gives fast feedback because the tests at that level don't go through the user interface (UI). The top layer, mostly workflow tests through the UI, tends to give the slowest feedback. The one change that we made to Mike Cohn's original pyramid was the little cloud on the top representing manual and exploratory tests that cannot be fully automated. Many teams require some manual regression tests to supplement their automated checks; they should find ways to keep this feedback timely.